

# 实验报告：用调试器定位归并排序缺陷

| 项目   | 内容          |
|------|-------------|
| 课程名称 | 系统开发工具基础    |
| 实验主题 | Debugging   |
| 题目编号 | 第7题         |
| 学号   | 25020007191 |
| 姓名   | 周御基         |
| 实验日期 | 2026年8月31日  |

## 一、实验目的

- 运行存在缺陷的归并排序代码，确认结果错误
- 使用调试器（pdb / 等价调试器）在 `merge` 中设置断点，观察 `i`、`j`、`left[i]`、`right[j]`
- 通过最小修改修复缺陷（不得用 `sorted` 替代）
- 编写 `pytest` 测试（给定输入 + 含重复元素的列表）并保证通过

## 二、实验环境

- 操作系统：Ubuntu Linux（远程服务器 192.168.79.130）
- Python：3.13.3, `pytest` 9.1.1
- 调试器：python `pdb`（及 `sys.settrace` 事件观察）
- 远程连接：SSH（`paramiko`）

## 三、实验步骤与代码

### 步骤1：保存给定代码并运行，确认结果错误

创建 `~/q07/merge_sort.py`（含缺陷），运行给定输入：

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) ≤ 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

```
cd ~/q07 && python3 merge_sort.py
```

输出结果（错误）：

```
[1, 5, 4, 3, 4, 5, 6, 9]
```

期望结果应为排序后的 [1, 1, 2, 3, 4, 5, 6, 9]，实际输出明显错误，确认存在缺陷。

## 步骤2：使用调试器在 merge 中设置断点，观察变量

使用 pdb 在 merge 的缺陷行（第 8 行，含 right[i]）设置断点，并单步观察 i、j、left[i]、right[j]。以 merge([1,3],[1,4]) 为例：

```
p = pdb.Pdb()
p.cmdqueue.extend([
    "break /home/joe/q07/merge_sort.py:8", # 缺陷行设断点
    "continue",
    "p i", "p j", "p left[i]", "p right",
    "continue",
    "p i", "p j", "p right",
    "quit",
])
p.run("from merge_sort import merge; print('result=', merge([1,3],[1,4]))")
```

## 输出结果：

```
Breakpoint 1 at /home/joe/q07/merge_sort.py:8
i = 1
j = 0
left[i] = 3
right = [1, 4]
result = [1, 4, 3, 4]
```

## 断点观察结论：

- 当 `i=1, j=0` 命中 `else` 分支时, `left[i] = 3`, 而 `right[j] = 1` (`right` 最小值为 1)。
- 但代码写的是 `right[i]`, 即取 `right[1] = 4`, **取错了数组下标/元素**, 导致把 4 排到了 1 前面。
- 因此 `merge([1,3],[1,4])` 得到 `[1, 4, 3, 4]` (应为 `[1, 1, 3, 4]`)。

用程序化 trace 进一步印证 (每次进入 `else` 分支时打印)：

```
[merge] i=1 j=0 left[i]=3 right[j]=1 (line 8) ← else 分支
```

## 步骤3：最小修复缺陷

把 `else` 分支中的 `right[i]` 改为 `right[j]` (与 `j += 1` 保持一致)：

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1 # 修复:right[j] 而非 right[i]
    return result + left[i:] + right[j:]
```

运行修复后的代码：

```
python3 merge_sort.py
```

输出结果 (正确)：

```
[1, 1, 2, 3, 4, 5, 6, 9]
```

## 步骤4: 编写 pytest 测试并运行

创建 `~/q07/test_merge_sort.py` , 包含给定输入与含重复元素的列表两个测试:

```
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    assert merge_sort([4, 2, 4, 2, 4, 2]) == [2, 2, 2, 4, 4, 4]

def test_single_and_empty():
    assert merge_sort([]) == []
    assert merge_sort([7]) == [7]
```

```
pytest test_merge_sort.py -v
```

输出结果:

```
test_merge_sort.py::test_given_input PASSED
test_merge_sort.py::test_duplicate_elements PASSED
test_merge_sort.py::test_single_and_empty PASSED
===== 3 passed in 0.10s =====
```

## 四、实验结果

### 4.1 结果展示

- 修复前: `[1, 5, 4, 3, 4, 5, 6, 9]` (错误)
- 修复后: `[1, 1, 2, 3, 4, 5, 6, 9]` (正确)
- pytest: 3 项测试全部通过 (含给定输入、重复元素、空/单元素)

## 4.2 结果分析

1. **缺陷定位**: 断点观察表明, else 分支在需要取出 `right` 中较小元素时, 误用了 `left[i]` 对应的索引访问 `right[i]`, 导致取到错误元素, 破坏了归并的稳定性与正确顺序。
2. **根因**: `right[i]` 应为 `right[j]`。当 `left[i] > right[j]` 时, 应把 `right[j]` (两数组中当前较小者) 放入结果并递增 `j`。
3. **最小修复**: 仅改一处下标, 未重写算法、未用 `sorted`, 符合"最小修复"要求。
4. **测试覆盖**: 给定输入覆盖基本排序, 重复元素用例验证  $\leq$  的稳定性 (相等元素按序保留), 空/单元素用例覆盖递归边界。

## 五、实验总结

---

通过本次实验, 我掌握了以下技能:

1. 使用调试器 (pdb) 设置断点并观察局部变量, 定位逻辑缺陷
  2. 通过跟踪 `i`、`j`、`left[i]`、`right[j]` 发现数组下标越界/取错元素的根因
  3. 用最小修改修复算法缺陷, 保持排序算法不变
  4. 用 `pytest` 编写覆盖给定输入、重复元素等场景的回归测试并保证通过
- 

实验中的脚本文件:

1. `Scripts/q07/merge_sort.py` (含修复后版本)
2. `Scripts/q07/test_merge_sort.py`
3. `Scripts/q07/debug_q07.py`、`Scripts/q07/pdb_drive_q07.py` (调试演示)